

Modaily

Dokumentacja Techniczna

Platforma E-Commerce — MERN Stack

MongoDB · Express · React · Node.js · Stripe · Redis · Cloudinary
2024

1. Opis projektu

Modaily to pełnostackowa aplikacja e-commerce zbudowana w oparciu o stos MERN, umożliwiająca przeglądanie produktów modowych według kategorii, zarządzanie koszykiem, płatności przez Stripe oraz kompleksowy panel administratora z analityką sprzedaży.

Główne funkcje:

- Przeglądanie produktów w 6 kategoriach (jeansy, t-shirty, buty, okulary, kurtki, garnitury, torby)
- Karuzela wyróżnionych produktów na stronie głównej — wyniki cachowane w Redis
- Koszyk z zarządzaniem ilością i dynamicznym przeliczaniem sumy
- Płatność przez Stripe Checkout z obsługą kodów rabatowych
- Automatyczne generowanie kuponu 10% po zakupie powyżej \$200
- Panel admina: tworzenie produktów (Cloudinary), lista z toggle "wyróżniony", usuwanie, analityka
- JWT: access token (15 min) + refresh token (7 dni) w ciasteczkach HTTP-only
- Automatyczne odnawianie tokena przez interceptor Axios
- Turborepo — monorepo zarządzający frontendem i backendem

2. Stos technologiczny

Warstwa	Technologia
Frontend	React 18, Vite, Tailwind CSS, Framer Motion
State Management	Zustand
Routing	React Router DOM v6
UI	Lucide React, React Hot Toast, Recharts, React Confetti
Backend	Node.js 20, Express 5
Baza danych	MongoDB 7 (Mongoose)
Cache	Upstash Redis (HTTP, @upstash/redis)
Autoryzacja	JWT (HTTP-only cookie), bcryptjs
Płatności	Stripe Checkout

Zdjęcia	Cloudinary
Konteneryzacja	Docker, Docker Compose
Monorepo	Turborepo

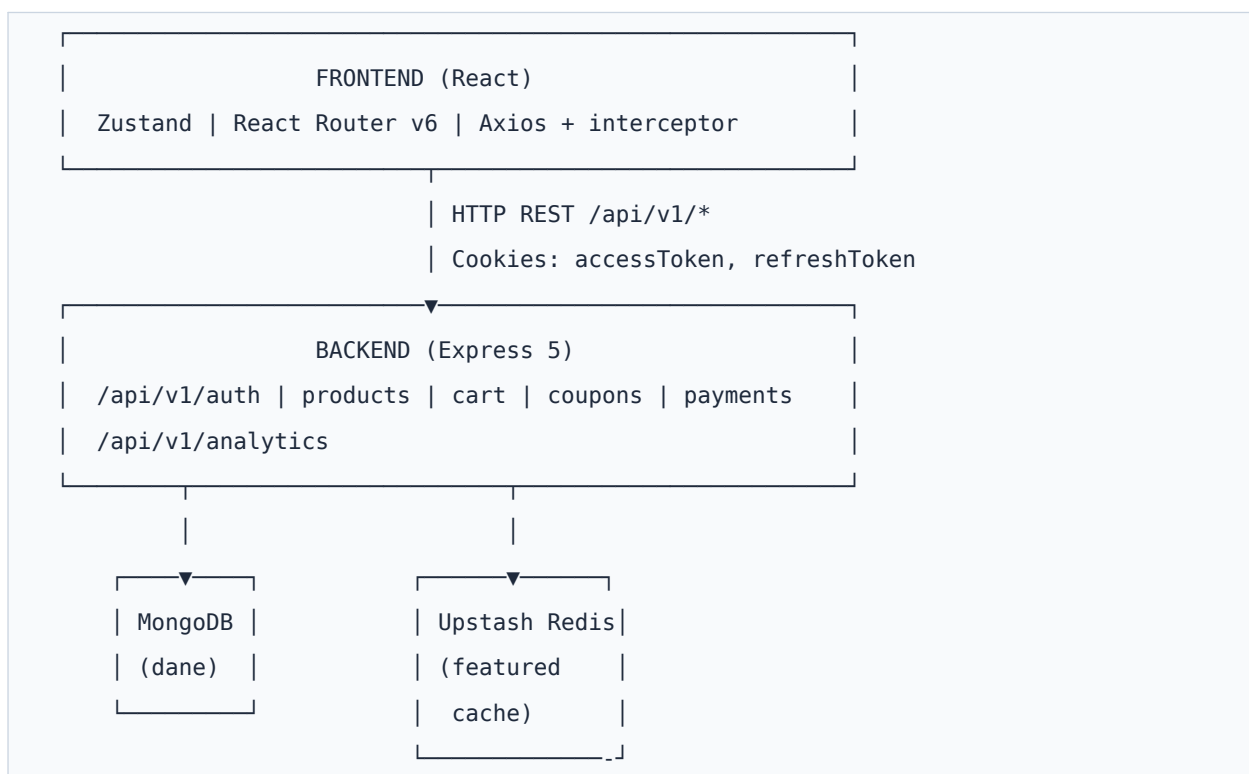
3. Struktura projektu

```
14-e-commerce/
├─ backend/
│   └─ controllers/
│       │   └─ auth.controller.js      # Signup, login, logout, refresh, profil
│       │   └─ product.controller.js   # CRUD, featured, kategorie, cache Redis
│       │   └─ cart.controller.js      # Pobierz, dodaj, usuń, aktualizuj ilość
│       │   └─ coupon.controller.js    # Pobierz kupon, walidacja kodu
│       │   └─ payment.controller.js   # Stripe Checkout, potwierdzenie płatności
│       │   └─ analytics.controller.js # Statystyki użytkowników, produktów, zamówień
│       └─ models/
│           │   └─ user.model.js        # name, email, password, cartItems[], role
│           │   └─ product.model.js     # name, description, price, image, isFeatured
│           │   └─ order.model.js       # user, products[], totalAmount, stripeSessionId
│           │   └─ coupon.model.js      # code, discountPercentage, expirationDate, userId
│       └─ middleware/
│           └─ auth.middleware.js       # protectRoute, adminRoute
│   └─ lib/
│       │   └─ db.js / redis.js / stripe.js / cloudinary.js
│       └─ server.js                   # Express, port 5001
├─ frontend/src/
│   └─ pages/
│       │   └─ HomePage.jsx / CategoryPage.jsx / CartPage.jsx
│       │   └─ AdminPage.jsx / LoginPage.jsx / SignUpPage.jsx
│       │   └─ PurchaseSuccessPage.jsx / PurchaseCancelPage.jsx
│   └─ components/
│       │   └─ Navbar.jsx / ProductCard.jsx / CartItem.jsx
│       │   └─ OrderSummary.jsx / GiftCouponCard.jsx / FeaturedProducts.jsx
│       └─ AnalyticsTab.jsx / CreateProductForm.jsx / ProductsList.jsx
```

```
| └─ stores/
|     └─ useUserStore.js / useCartStore.js / useProductStore.js
├─ Dockerfile          # Produkcja: build frontend + uruchom backend
├─ backend.Dockerfile   # Sam backend
├─ frontend.Dockerfile  # Frontend z Nginx (lokalnie)
├─ docker-compose.yml
└─ turbo.json
```

4. Architektura aplikacji

Schemat komunikacji



Przepływ autoryzacji

- Rejestracja/logowanie → generowany accessToken (15 min) i refreshToken (7 dni)
- Oba tokeny ustawiane jako ciasteczka HTTP-only — niedostępne przez JavaScript (ochrona XSS)
- refreshToken przechowywany dodatkowo w Redis z TTL 7 dni

- Interceptor Axios wykrywa odpowiedź 401 i automatycznie wywołuje /auth/refresh-token
- Jeśli odświeżenie się nie powiedzie — użytkownik jest automatycznie wylogowywany

5. Instalacja i uruchomienie

Wymagania wstępne

- Node.js 20+
- MongoDB (lokalne lub Atlas)
- Konto Upstash Redis
- Konto Stripe
- Konto Cloudinary

Lokalny development

```
# Zainstaluj wszystkie zależności (Turborepo workspace)
npm install

# Uruchom backend i frontend równolegle
npm run dev

# Backend → http://localhost:5001
# Frontend → http://localhost:5173
```

Plik backend/.env.local

```
MONGO_URI=mongodb+srv://...
ACCESS_TOKEN_SECRET=<min_32_znaki>
REFRESH_TOKEN_SECRET=<min_32_znaki>
UPSTASH_REDIS_REST_URL=https://...upstash.io
UPSTASH_REDIS_REST_TOKEN=...
STRIPE_SECRET_KEY=sk_live_...
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...
CLIENT_URL=http://localhost:5173
PORT=5001
```

```
NODE_ENV=development
```

Plik frontend/.env

```
VITE_STRIPE_PUBLISHABLE_KEY=pk_live_...
```

Docker (lokalnie)

```
cp .env.example .env # uzupełnij dane
docker compose up --build
# Aplikacja → http://localhost
```

6. API — dokumentacja endpointów

Autoryzacja — /api/v1/auth

Metoda + Endpoint	Auth	Opis
POST /signup	Nie	Rejestracja — name, email, password
POST /login	Nie	Logowanie — ustawia ciasteczka JWT
POST /logout	Tak	Wylogowanie — czyści ciasteczka i Redis
POST /refresh-token	Nie	Odnów access token z refresh cookie
GET /profile	Tak	Pobierz aktualnie zalogowanego użytkownika

Produkty — /api/v1/products

Metoda + Endpoint	Auth	Opis
GET /	Admin	Wszystkie produkty
GET /featured	Nie	Wyróżnione produkty (cache Redis)
GET /category/:category	Nie	Produkty według kategorii
GET /recommendations	Nie	3 losowe produkty
POST /	Admin	Utwórz produkt (upload Cloudinary)

PATCH /:id	Admin	Przełącz isFeatured, wyczyść cache
DELETE /:id	Admin	Usuń produkt

Koszyk — /api/v1/cart (wymagana autoryzacja)

Metoda	Opis
GET /	Pobierz produkty z koszyka
POST /	Dodaj produkt do koszyka
PUT /:id	Aktualizuj ilość produktu
DELETE /	Usuń produkt z koszyka

Kupony i Płatności

Endpoint	Opis
GET /api/v1/coupons	Pobierz aktywny kupon użytkownika
POST /api/v1/coupons/validate	Waliduj kod kuponu
POST /api/v1/payments/create-checkout-session	Utwórz sesję Stripe Checkout
POST /api/v1/payments/checkout-success	Potwierdź płatność, zapisz zamówienie
GET /api/v1/analytics	Statystyki (tylko admin)

7. Modele danych

User

```
{ name, email, password (bcrypt), role: "customer"|"admin",
  cartItems: [{ product: ObjectId, quantity }] }
```

Product

```
{ name, description, price, image (Cloudinary URL),
  category, isFeatured: Boolean }
```

Order

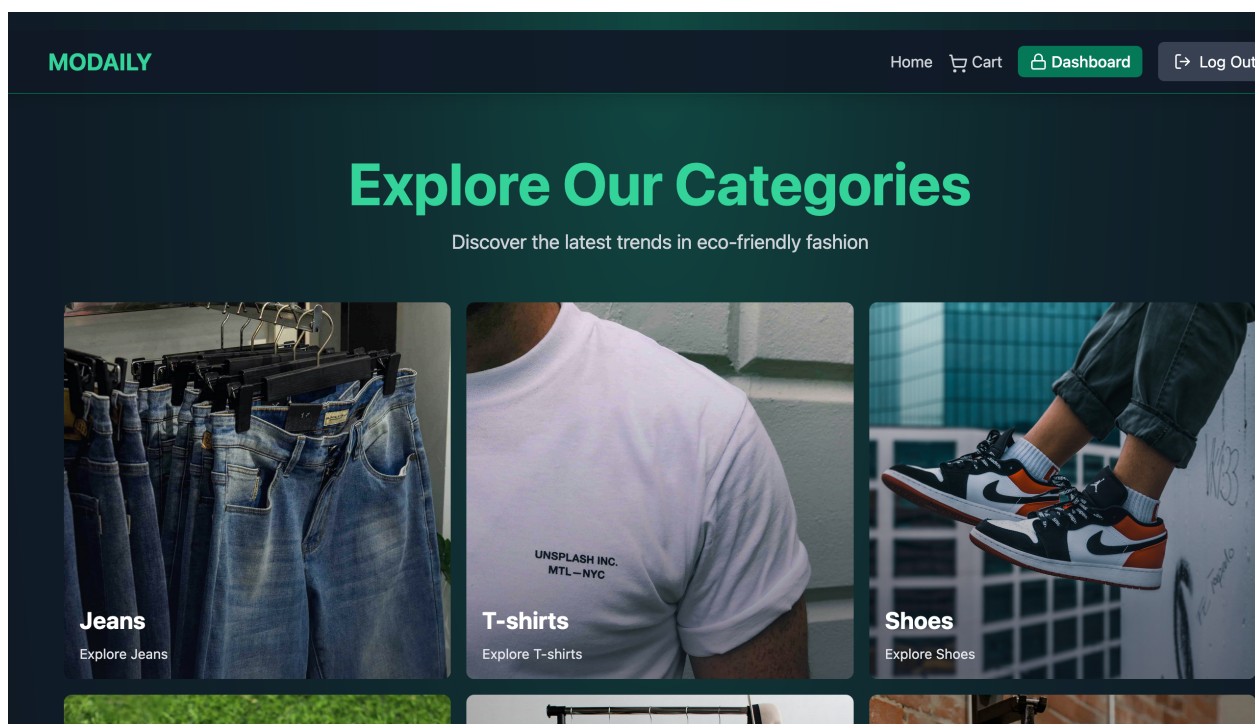
```
{ user: ObjectId, products: [{ product, quantity, price }],
  totalAmount, stripeSessionId }
```

Coupon

```
{ code, discountPercentage, expirationDate,
  isActive: Boolean, userId: ObjectId }
```

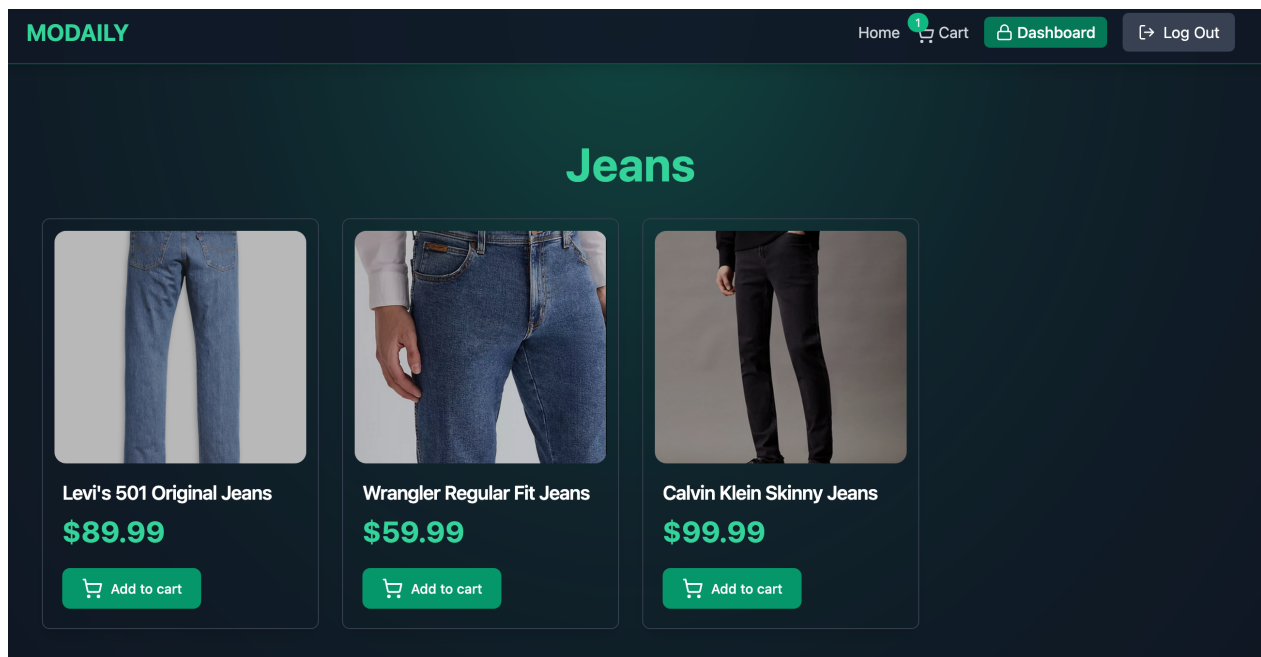
8. Podgląd aplikacji

Strona główna — kategorie



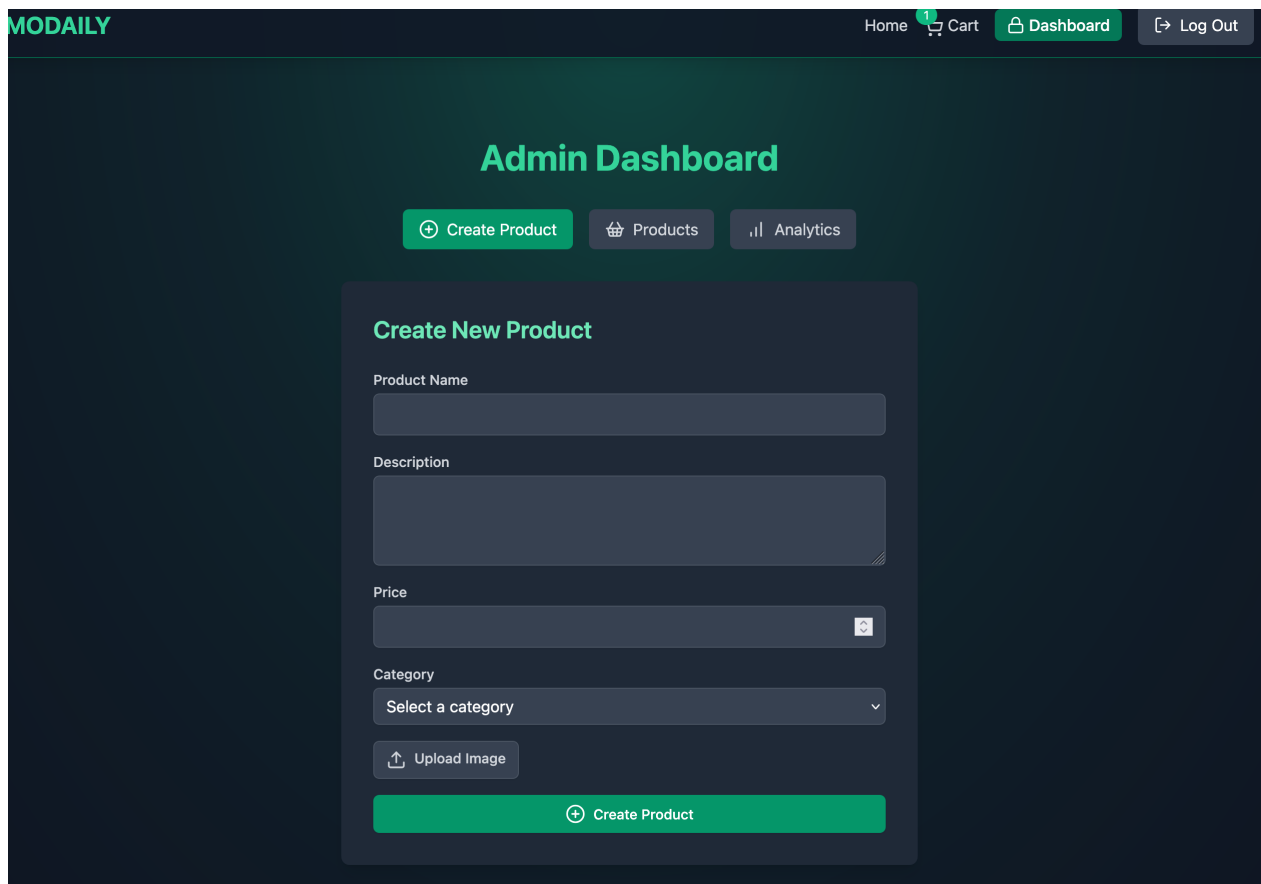
Rys. 1 — Strona główna z siatką kategorii produktów

Strona kategorii



Rys. 2 — Lista produktów w kategorii z kartami i przyciskiem "Dodaj do koszyka"

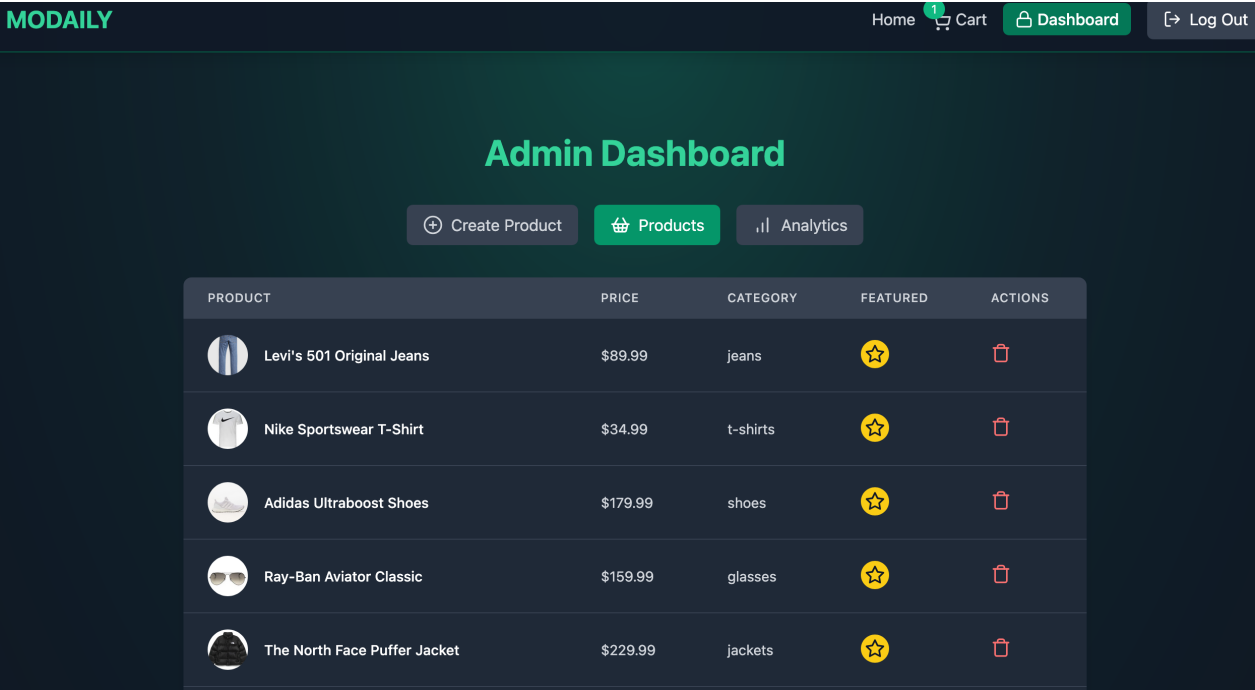
Panel admina — tworzenie produktu



The screenshot displays the 'Admin Dashboard' of the Modaily application. The dashboard has a dark theme with a teal accent color. At the top, there is a navigation bar with links for 'Home', 'Cart' (with a notification badge), 'Dashboard' (active), and 'Log Out'. Below the navigation bar, the 'Admin Dashboard' title is centered. Underneath the title, there are three buttons: 'Create Product' (teal), 'Products' (dark grey), and 'Analytics' (dark grey). The main content area features a 'Create New Product' form. The form includes the following fields: 'Product Name' (text input), 'Description' (text area), 'Price' (text input with a currency dropdown), 'Category' (dropdown menu with 'Select a category' as the placeholder), and an 'Upload Image' button. At the bottom of the form is a large teal 'Create Product' button.

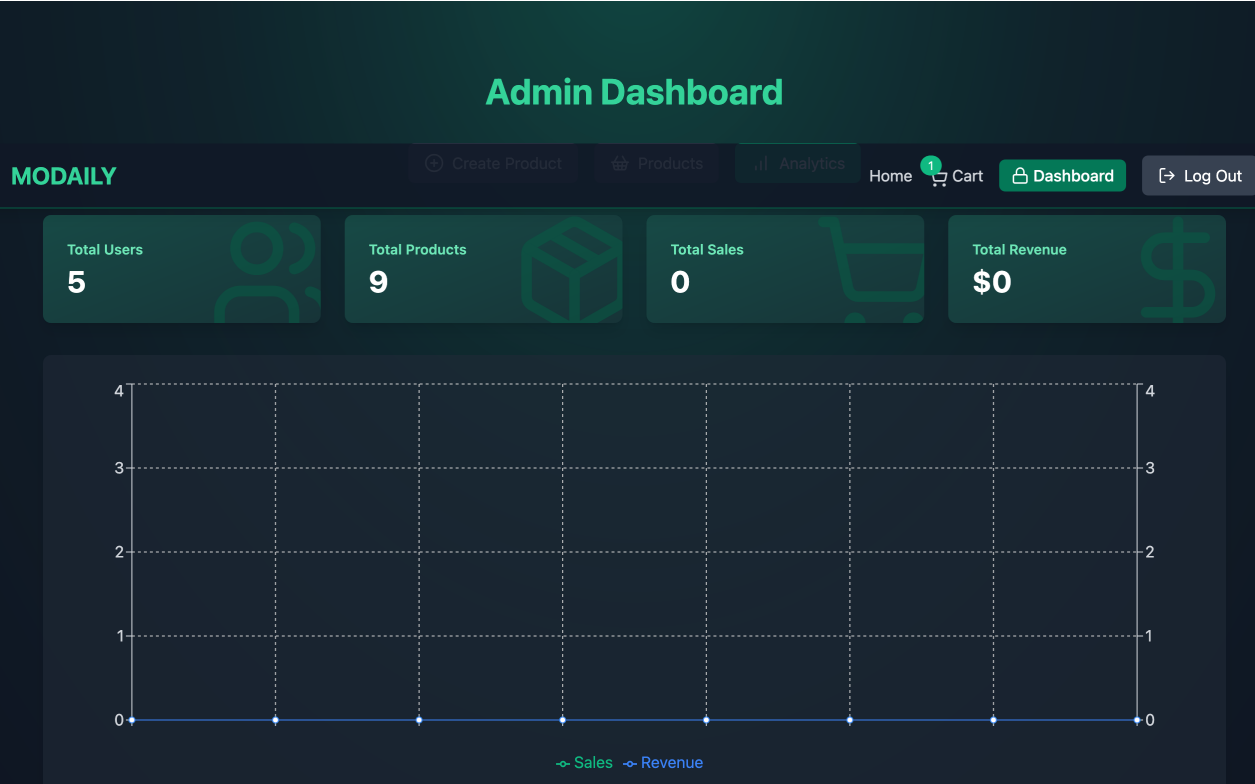
Rys. 3 — Formularz tworzenia produktu z upload zdjęcia do Cloudinary

Panel admina — lista produktów



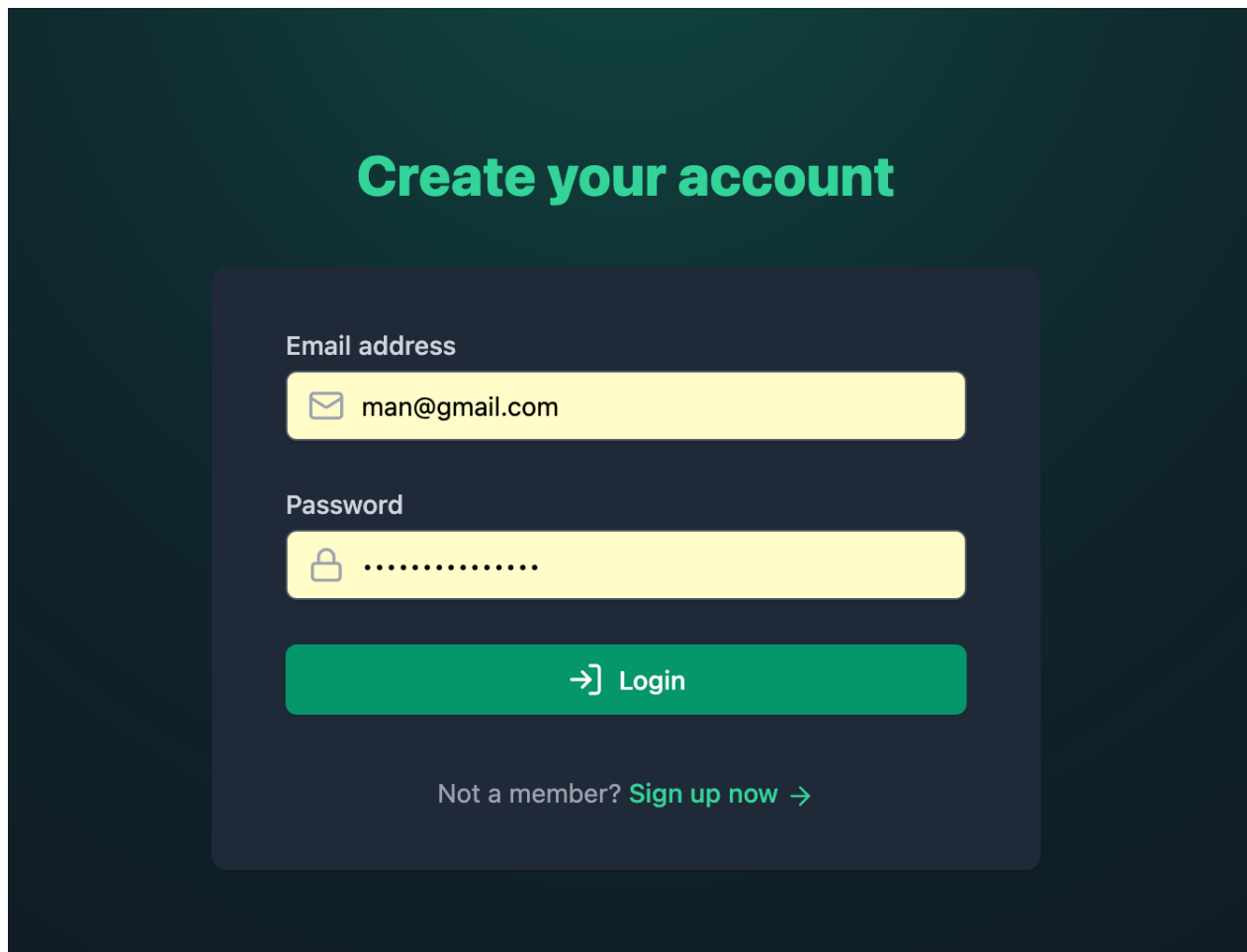
Rys. 4 — Tabela produktów z toggle "wyróżniony" i opcją usunięcia

Panel admina — analityka



Rys. 5 — KPI (użytkownicy, produkty, sprzedaż, przychód) i wykres dzienny

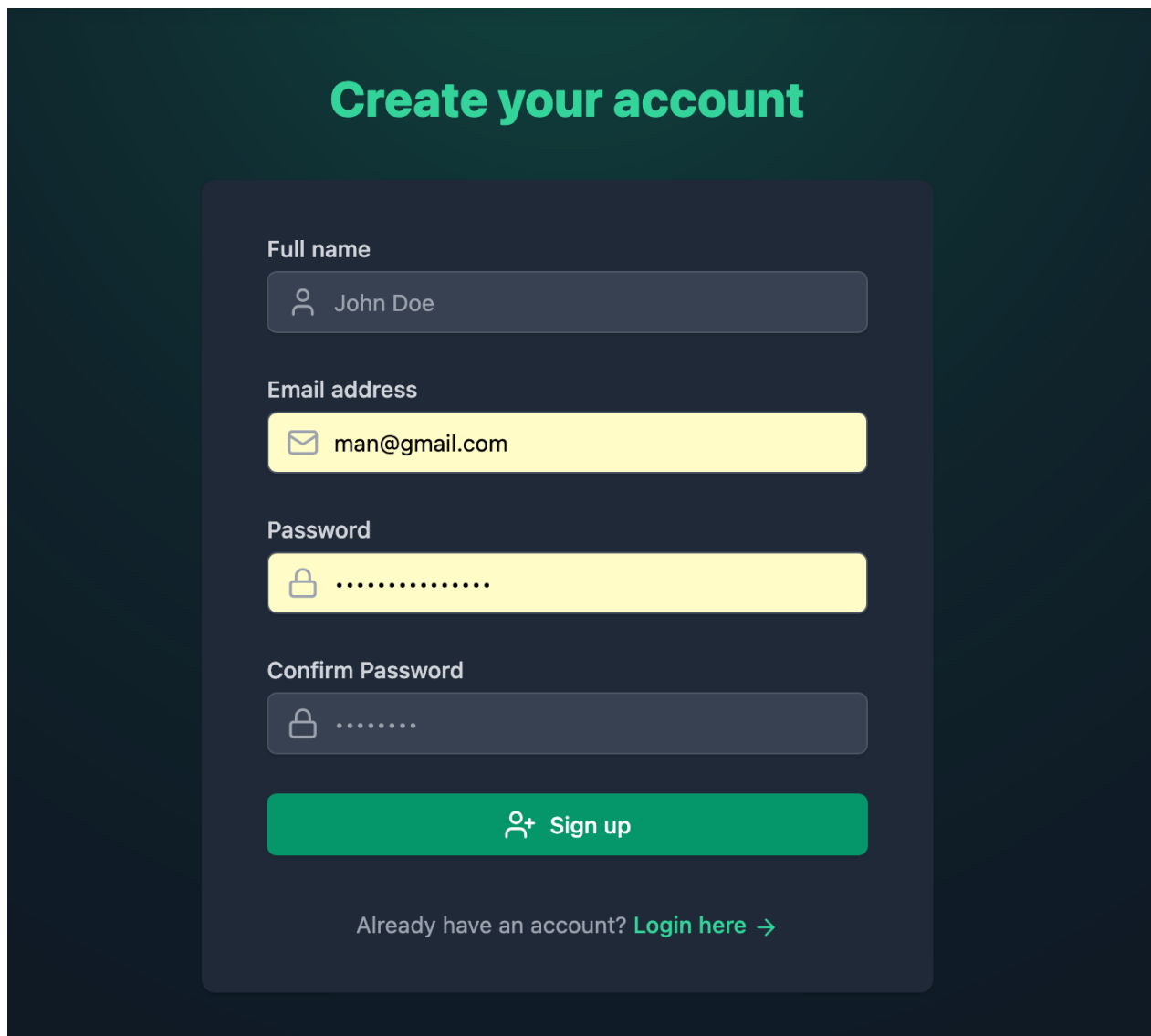
Logowanie



The screenshot shows a login interface with a dark teal background. At the top, the text "Create your account" is displayed in a large, bold, light green font. Below this, a dark gray rounded rectangle contains the login fields. The first field is labeled "Email address" and contains the text "man@gmail.com" next to an envelope icon. The second field is labeled "Password" and contains a series of dots next to a lock icon. Below these fields is a large green button with the text "→] Login". At the bottom of the gray rectangle, there is a link that says "Not a member? Sign up now →".

Rys. 6 — Formularz logowania z powiadomieniami toast

Rejestracja



Create your account

Full name

Email address

Password

Confirm Password

Already have an account? [Login here](#) →

Rys. 7 — Formularz rejestracji z walidacją zgodności haseł

9. System kuponów

- Każdy użytkownik może posiadać co najwyżej jeden aktywny kupon jednocześnie
- Kupony są walidowane po stronie serwera względem userId przed płatnością
- Nowy kupon 10% generowany automatycznie przy zakupie powyżej \$200
- Użyte kupony są dezaktywowane po pomyślnej płatności Stripe
- Kod kuponu przekazywany przez metadane sesji Stripe i weryfikowany w webhook

10. Docker i wdrożenie

Lokalnie (Docker Compose)

Trzy serwisy: MongoDB, backend Express, frontend Nginx. Nginx proxy kieruje /api/* do backendu.

```
cp .env.example .env
docker compose up --build
```

Render (produkcja)

Jeden serwis Web Service z głównym Dockerfile — buduje frontend Vite, a Express serwuje zarówno API jak i statyczne pliki dist.

- Typ serwisu: Web Service
- Runtime: Docker
- Katalog główny: 14-e-commerce
- Wszystkie zmienne środowiskowe ustawione w panelu Render
- VITE_STRIPE_PUBLISHABLE_KEY jako Build Argument (wbudowany przez Vite)

11. Znane problemy

Problem	Opis
Brak rate limitingu na /auth/*	Endpointy logowania i rejestracji narażone na brute-force
ADMIN_ROLE env var nieużywany	Rola admina przypisywana ręcznie w MongoDB, env var jest zbędny
dotenv + Render	dotenv.config({ path: ".env.local" }) wycisza się bez pliku — działanie popraw
Brak walidacji inputów	Brak walidacji (np. Joi/Zod) po stronie serwera dla formularzy